

Lab 10 Methods II

Objective

- To practically familiarize student with how to write the common code once and reuse it without rewriting it using the concept of Methods.

Current Lab Learning Outcomes (LLO)

As a second Lab on Chapter 6, this lab continues practicing the Chapter 6 theory with students, with using idea of methods. By completion of the lab the students should be able to

1. define their own methods
2. invoke (call) a method (method invocation)
3. passing parameters by values
4. how a method may return a value
5. how to write and use overloading methods

Lab Requirements

NetBeans IDE 8.0.1 or higher

Lab Assessment/Activities

Practice Activity with Lab Instructor (10 minutes)

(Sum series) Write a method to compute the following series:

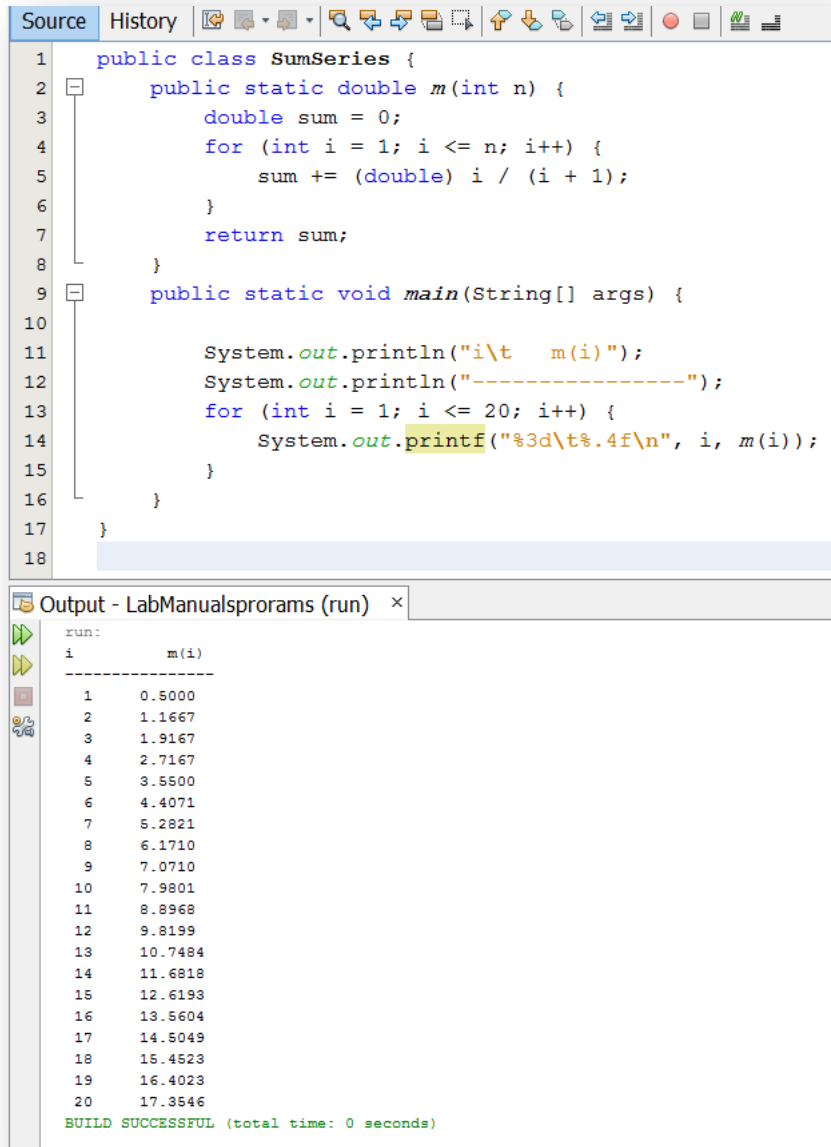
$$m(i) = \frac{1}{2} + \frac{2}{3} + \dots + \frac{i}{i+1}$$

Write a test program that displays the following table:

i	m(i)
1	0.5000
2	1.1667
...	
19	16.4023
20	17.3546

Solution:

1. Open NetBeans and create a new project
2. Create a new java main class and write its name as **SumSeries**
3. Write the following code inside the class, Figure (1).



```
1 public class SumSeries {
2     public static double m(int n) {
3         double sum = 0;
4         for (int i = 1; i <= n; i++) {
5             sum += (double) i / (i + 1);
6         }
7         return sum;
8     }
9     public static void main(String[] args) {
10
11         System.out.println("i\t    m(i)");
12         System.out.println("-----");
13         for (int i = 1; i <= 20; i++) {
14             System.out.printf("%3d\t%.4f\n", i, m(i));
15         }
16     }
17 }
18
```

Output - LabManualsprograms (run) x

```
run:
i      m(i)
-----
1      0.5000
2      1.1667
3      1.9167
4      2.7167
5      3.5500
6      4.4071
7      5.2821
8      6.1710
9      7.0710
10     7.9801
11     8.8968
12     9.8199
13     10.7484
14     11.6818
15     12.6193
16     13.5604
17     14.5049
18     15.4523
19     16.4023
20     17.3546
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figure 1: The program to compute a series based on methods

Short Exercise (5 minutes)

(Sum series) Rewrite the above method to compute the following series:

$$m(i) = \frac{1}{2} - \frac{2}{3} + \dots + \frac{i}{i+1}$$

Individual Activities: (60 minutes)

1. **(Occurrences of a specified character)** Write a method that finds the number of occurrences of a specified character in a string using the following header:

```
public static int count(String str, char a)
```

For example, `count("Welcome", 'e')` returns 2. Write a test program that prompts the user to enter a string followed by a character and displays the number of occurrences of the character in the string.

2. **(Number of days in a year)** Write a method that returns the number of days in a year using the following header:

```
public static int numberOfDaysInAYear(int year)
```

Write a test program that displays the number of days in year from 2000 to 2020.

Hint: You need to check whether the year is a leap year or not. If it is a leap year, set number of days to 366 else 365.

3. **(The MyTriangle class)** Create a class named **MyTriangle** that contains the following two methods:

```
/** Return true if the sum of any two sides is greater than the third side. */
```

```
public static boolean isValid( double side1, double side2, double side3)
```

```
/** Return the area of the triangle. */
```

```
public static double area( double side1, double side2, double side3)
```

Write a test program that reads three sides for a triangle and computes the area if the input is valid. Otherwise, it displays that the input is invalid. The formula for computing the area of a triangle is

$$s = (\text{side1} + \text{side2} + \text{side3})/2$$

$$\text{area} = \sqrt{s(s - \text{side1})(s - \text{side2})(s - \text{side3})}$$

4. (**Mersenne prime**) A prime number is called a Mersenne prime if it can be written in the form $2^p - 1$ for some positive integer p . (if p is prime and result of $2^p - 1$ also is prime, then this result called Mersenne prime). Write a program that finds all Mersenne primes with $p \leq 31$ and displays the output as follows:

p	$2^p - 1$
2	3
3	7
5	31
7	127
13	8191
17	131071
19	524287
31	2147483647

Hint: It is preferable to implement the power method by yourself. you can use the following header

```
public static long pow(long base, long exponent)
```

5. (**Format an integer**) Write a method with the following header to format the integer with the specified width.

```
public static String format(int number, int width)
```

The method returns a string for the number with one or more prefix 0s. The size of the string is the width. For example, `format(34, 4)` returns `0034` and `format(34, 5)` returns `00034`. If the number is longer than the width, the method returns the string representation for the number. For example, `format(34, 1)` returns `34`.

Write a test program that prompts the user to enter a number and its width and displays a string returned by invoking `format(number, width)`.

Lab Description

- A method is a collection of statements grouped together to perform an operation.
- When you have multiple methods with the SAME method name this is called method overloading, Figure (2) is an example of overloading method.
- However, the method signature is different
- This means that the number of parameters will be different, or the data types of the parameters will be different

```
1 public class TestMethodOverloading {
2     /** Main method */
3     public static void main(String[] args) {
4         // Invoke the max method with int parameters
5         System.out.println("The maximum of 3 and 4 is "
6             + max(3, 4));
7
8         // Invoke the max method with the double parameters
9         System.out.println("The maximum of 3.0 and 5.4 is "
10            + max(3.0, 5.4));
11
12        // Invoke the max method with three double parameters
13        System.out.println("The maximum of 3.0, 5.4, and 10.14 is "
14            + max(3.0, 5.4, 10.14));
15    }
16
17    /** Return the max of two int values */
18    public static int max(int num1, int num2) {
19        if (num1 > num2)
20            return num1;
21        else
22            return num2;
23    }
24
25    /** Find the max of two double values */
26    public static double max(double num1, double num2) {
27        if (num1 > num2)
28            return num1;
29        else
30            return num2;
31    }
32
33    /** Return the max of three double values */
34    public static double max(double num1, double num2, double num3) {
35        return max(max(num1, num2), num3);
36    }
37 }
```

Figure 2: The program to find the max number using overloading methods

Extra (supplementary) Materials (if any)

None